

Week 1

You need to understand the “Essence difficulty” and the “Accidental difficulty”.

Essential Difficulty (The "Essence")

Essential difficulty represents the intrinsic complexity of the problem you are trying to solve. It is the core logic, data relationships, and business rules that must exist for the software to do its job. Even if you had a flawless programming language and zero technical friction, this difficulty would remain because it is part of the problem itself.

Brooks argues that software is inherently difficult because of four essential properties:

- **Complexity:** The sheer number of moving parts, states, and variables that must interact without contradiction.
- **Conformity:** Software must conform to messy, non-negotiable human institutions, legacy systems, and real-world rules, which rarely follow mathematical logic.
- **Changeability:** Unlike physical structures, software is under constant pressure to evolve because it sits at the center of changing human processes.
- **Invisibility:** Software is abstract and cannot be easily visualized in space, making it incredibly hard for the human mind to map completely.

Accidental Difficulty (The "Accident")

Accidental difficulty refers to the hurdles introduced by our chosen implementation details, tools, languages, and development environments. It is the friction we encounter while trying to translate our conceptual solution into working code.

Examples of accidental difficulty include:

- Struggling with memory management or manual pointer allocation.
- Dealing with obscure compiler errors, rigid syntax, or missing library support.
- Configuring complex build tools, deployment pipelines, or boilerplate framework code.
- Optimizing slow execution speeds caused by inefficient underlying execution environments.

Key Differences

The easiest way to contrast the two is to ask: *If my development tools were completely perfect, would this problem go away?* If yes, it's accidental. If no, it's essential.

Important notes

- To identify essence difficulty, find if one of the four essential properties exist: Complexity, Conformity, Changeability, Invisibility.
- Tutorial questions are important. Question 1, 3, 6, 9. Understand the Q&A, not memorized it.

Week 2

Cohesion and Coupling are the twin pillars of structured design, and your goal as a developer is always to achieve high cohesion and low coupling.

The easiest way to remember the distinction is:

- Cohesion is about how focused a single module is. (Internal focus)
- Coupling is about how dependent different modules are on each other. (External focus)

Cohesion

Cohesion measures how closely related and focused the responsibilities inside a single module, class, or component are.

Think of a module as a kitchen drawer. If you have a drawer labeled "Silverware" that only contains forks, knives, and spoons, it has high cohesion. Everything in it belongs together and serves a single purpose. If you have a "Junk Drawer" containing batteries, old receipts, a screwdriver, and half-eaten candy, it has low cohesion.

- High Cohesion (Good): A class does one thing and one thing only (aligning with the Single Responsibility Principle). If you have a `PaymentProcessor` class, it handles authorization, capturing, and refunds. It doesn't print PDFs or parse HTML.
- Low Cohesion (Bad): A utility module (like `Utils` or `Manager`) that handles database connections, user validation, and email formatting all in one place. It is hard to understand, maintain, or reuse.

Coupling

Coupling measures the degree of interdependence between two or more modules. It asks: If I change code in Module A, will it break Module B?

- Loose/Low Coupling (Good): Modules interact through clean, well-defined interfaces without needing to know the internal details of how the other works. They are independent. If you swap out your Postgres database wrapper for a MongoDB wrapper, the user interface code shouldn't even notice.

- **Tight/High Coupling (Bad):** Modules are deeply intertwined. Module A directly reads and modifies the private internal variables of Module B. If you change a single line of code in the database layer, five unrelated UI components suddenly crash. This is often called "spaghetti code."

Single Responsibility Principle

The Single Responsibility Principle (SRP) is the "S" in the famous SOLID design principles.

The classic definition states: A class should have one, and only one, reason to change.

The Benefits of SRP

Adhering to SRP brings several massive advantages to a codebase, particularly as a system scales:

- Higher Maintainability
- Reduced Risk of Regressions (Domino Effects)
- Easier Testing
- Improved Reusability

The problems of code violating SRP

- More difficult to understand and maintain
- Often clutter with codes and difficult to test and debug
- Difficult to navigate and hard to find a specific code
- Reduce reusability
- Etc

Open/Closed Principle

The Open/Closed Principle (OCP) is the "O" in the SOLID design principles.

The principle states: Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.

In plain English: You should be able to add new features or behaviors to your code without changing a single line of the code you already wrote.

We close the service to modification by opening it up to extension through a shared contract (an interface).

Inheritance vs Interface in OCP.

While we can achieve OCP using inheritance, relying heavily on inheritance to achieve OCP introduces a major risk known as the Fragile Base Class Problem.

When you inherit from a class, you break encapsulation. Your subclass is tightly coupled to the internal implementation details of the parent class. If a developer modifies a private behavior or structure in the base class to fix a bug, they can unknowingly trigger a domino effect that breaks dozens of inherited subclasses.

Week 3&4

Factory Pattern, Decorator Pattern, State Pattern.

Week 5

Application Frameworks (Technical Reuse)

An Application Framework is a cohesive set of concrete and abstract classes that provides a reusable, skeletal architecture for a specific type of software application.

The defining characteristic of a framework is **Inversion of Control (IoC)**. Instead of your code calling a library, the framework owns the main execution flow and calls *your* custom code at specific hooks or extension points.

- **How it works:** It handles the repetitive, infrastructural accidental difficulties (like routing, dependency injection, database connection pooling, or security parsing). You write only the essential business logic.
- **Examples:** **Spring Boot** (Java) for enterprise web applications, **ROS2/Nav2** (C++) for autonomous robotics navigation, or **Django** (Python).

Software Product Lines (Strategic Domain Reuse)

A Software Product Line (SPL) is a strategic, engineering-wide approach where a software development organization creates a portfolio of distinct but highly related systems from a shared set of core software assets.

Instead of building completely separate systems for different clients, an SPL focuses on identifying a **core modules** (common features shared by all variants) and explicit **variation points/specialization** (features that differ based on customer needs, hardware constraints, or market segments).

- **How it works:** A warehouse automation provider might have a single software product line. Client A needs a 4D pallet shuttle system; Client B needs an Autonomous Mobile Robot (AMR) system. Both systems run on the exact same core warehouse control software, but different vision algorithms (e.g., YOLO node configurations) or hardware drivers are plugged into the predefined variation points.
- **Examples:** A software suite adapted for different vehicle dashboard configurations (economy vs. luxury models), or a core mobile banking app customized for 20 different regional banks.

COTS - Commercial Off-The-Shelf (Acquisition Reuse)

COTS involves buying or licensing third-party, pre-built software products from the commercial market and integrating them directly into your system, rather than developing them in-house.

With COTS, you treat the software as a black box. You have zero control over its internal code, meaning you must build adapters to bridge your system to the vendor's APIs when you integrate the COTS to your existing system.

- **How it works:** Instead of writing a custom accounting engine, payment gateway, or CRM from scratch, you buy access to an existing commercial solution and hook it up via APIs or data pipelines.
- **Examples:** Integrating **Interactive Brokers (IBKR) TWS API** for market data execution, using **Salesforce** for customer management, or using **Stripe** for handling credit card transactions.

Characteristics of the Strategies

Application Frameworks (The "Build with a Skeleton" Strategy)

- **Development Length: Medium.** It drastically accelerates the initial setup (saving weeks of writing infrastructure code), but because you are still writing the actual application logic from scratch, your overall development timeline depends on the complexity of your features.
- **Type of Project: Custom, highly proprietary software** where your business logic is unique, or cutting-edge systems that don't exist as pre-built commodities on the market (e.g., a proprietary AI-driven algorithmic trading platform or a unique robotics navigation pipeline).
- **Cost of the Project: Medium.** Most major frameworks are open-source (free licensing), so your main financial investment is paying for the engineering hours required to write the custom code.
- **Skills Required: High technical expertise.** Developers must thoroughly understand software design patterns, object-oriented or functional paradigms, and the specific lifecycle, annotations, and inversion-of-control rules of the chosen framework.

- **System Lifespan: Long (5 to 10+ years).** Because you own the source code, your team can maintain, patch, and refactor the system indefinitely, updating framework dependencies over time.
- **Level of Customization (Control): Maximum.** You have absolute control. The framework dictates the *structural boundaries*, but within those boundaries, you can write whatever logic you want and alter it at will.

Software Product Lines (The "Factory Platform" Strategy)

- **Development Length: Very Long initially, then ultra-short.** Building the core platform and mapping out all the "variation points" takes months or even years of upfront engineering before a single product can ship. However, once the platform is complete, spinning up a new variant for a new client takes only days or weeks.
- **Type of Project: Portfolios of deeply related software variants.** Best for B2B enterprise software suites, embedded systems across a family of consumer hardware devices, or localized versions of a core product (e.g., warehouse control software adapted for 15 different warehouse physical configurations).
- **Cost of the Project: Very High Upfront.** It requires heavy financial backing to fund a dedicated platform team that isn't immediately shipping client features. The massive return on investment (ROI) only triggers once you deploy your 4th, 5th, or 20th product variant.
- **Skills Required: Expert-level System Architecture.** This requires elite engineers who excel at domain analysis, loose coupling, abstract interfaces, and designing highly configuration-driven software that doesn't collapse into spaghetti code when modified for a new client.
- **System Lifespan: Very Long (10 to 15+ years).** An SPL represents a core company asset. The architecture is explicitly designed to evolve continuously as new variations are discovered in the market.
- **Level of Customization (Control): High internally, but highly disciplined.** Your organization has total control over the code, but developers must strictly respect the pre-defined variation points. You cannot allow ad-hoc, messy customizations for individual clients without breaking the integrity of the product line.

COTS - Commercial Off-The-Shelf (The "Acquisition" Strategy)

- **Development Length: Shortest (Days to Weeks).** The software is already written and tested. Your timeline is strictly bound by the time it takes to read the vendor's documentation, configure accounts, and write integration "glue code" (like API wrappers).
- **Type of Project: Commodity, auxiliary, or highly regulated utilities.** Subsystems that are critical to run the business but do not form your core competitive advantage (e.g., payment gateways, CRM tracking, user authentication, or enterprise cloud databases).

- **Cost of the Project: Predictable ongoing cost.** Initial implementation costs are very low, but you are locked into continuous subscription fees, usage-based pricing, or licensing costs (SaaS models) that scale up as your business grows.
- **Skills Required: Integration and API Expertise.** Developers do not need to know how to write complex underlying algorithms (e.g., how to build an encryption engine). Instead, they need strong skills in building robust adapters, data mapping, error handling for network failures, and parsing JSON/XML data payloads safely.
- **System Lifespan: Short to Medium (Dependent on Vendor).** Your system lifespan is tied to the vendor's roadmap. If the vendor deprecates an API, hikes prices significantly, goes out of business, or stops supporting a feature, you are forced to refactor your system to migrate to a competitor.
- **Level of Customization (Control): Very Low.** The software is a strict black box. You cannot alter the source code. If you need a custom feature that the vendor's settings or public APIs don't support, you are entirely out of luck unless you build complex, external workarounds.

Week 6

Fault Tree, probability calculation.

Finally: All tutorials are important.